

OAuth 2.0

Grundlagen des Protokolls und
beispielhafte Implementierung mit
Authentik Backend

Fabian Schuller

03.02.2025



Motivation

Jeder hat schon (wenn auch unwissend) OAuth benutzt.

OAuth ermöglicht Authentifizierung und Autorisierung über mehrere Services hinweg.

OAuth ist auch dann sicher, wenn man Teilkomponenten der Architektur nicht vertrauen kann.



Welcome back.

 Sign in with Google

 Sign in with Facebook

 Sign in with Apple

 Sign in with X

 Sign in with email

No account? [Create one](#)

Forgot email or trouble signing in? [Get help](#).

Click "Sign in" to agree to Medium's [Terms of Service](#) and acknowledge that Medium's [Privacy Policy](#) applies to you.

Inhalt

Der Vortrag hat das Ziel, das OAuth2.0 Protokoll in steigender Komplexität zu erklären.

Am Ende sollte die ZuhörerInnen in der Lage sein, OAuth in eigenen Anwendungen zu benutzen, und auf ein gesteigertes Bewusstsein für IT-Sicherheit im Alltag haben.

Grundlagen

- Motivation
- Terminologie
- Architektur
- Public und Confidential Clients

Tokens

- Opaque Tokens Vs. Json Web Tokens
- Access, Refresh, und ID Tokens

Flows

- Authorization
- Client Credentials
- Revokation
- Authorization mit PKCE (Proof Key for Code Exchange)

Angriffsvektoren

- CSRF (Cross-Site Request Forgery)
- Authorization Code Injection
- Phishing Attacken
- Replay Attacken

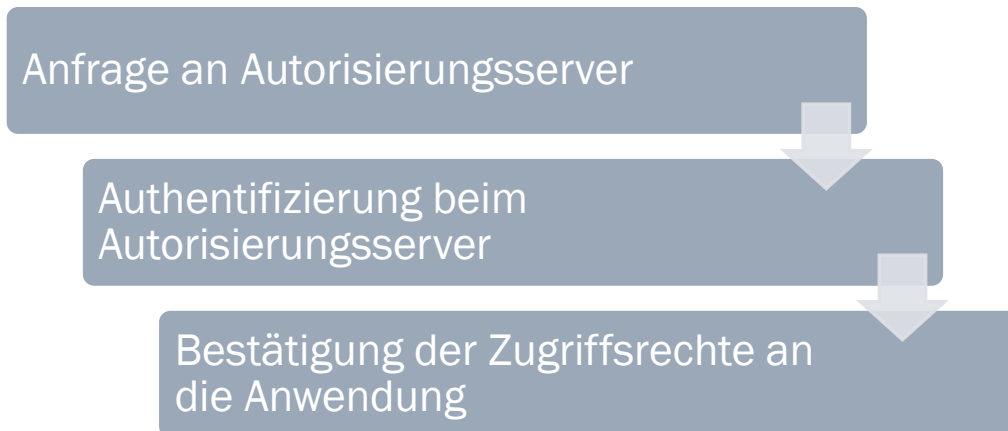
Fazit

- Eigene Implementation
- OAuth Anbieter

Terminologie

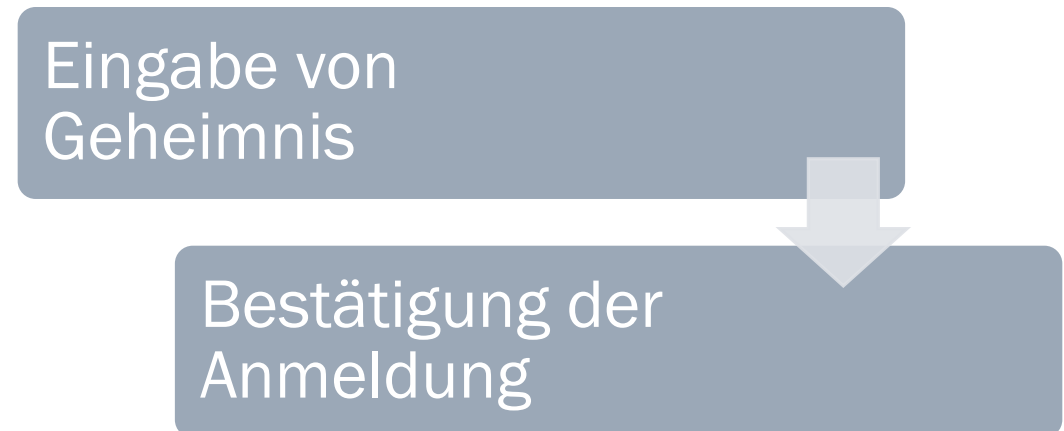
AUTORISIERUNG

- Ermächtigung einer 3. Partei
- Beispiel: „mit Google anmelden“
- Einschränkung von Zugriffsrechten möglich



AUTHENTIFIZIERUNG

- Anmeldung bei der Anwendung durch Passwort, biometrische Daten oder zweiten Faktor



Terminologie

Resource Owner

- Entität im Besitz der Ressourcen auf die zugegriffen werden sollen (meistens der Endbenutzer)
- Der Resource Owner interagiert mit einem **User Agent** (meistens ein Browser)

Client

- Die Anwendung, die im Namen des Resource Owners operiert (Die Entität, die *autorisiert* wird)

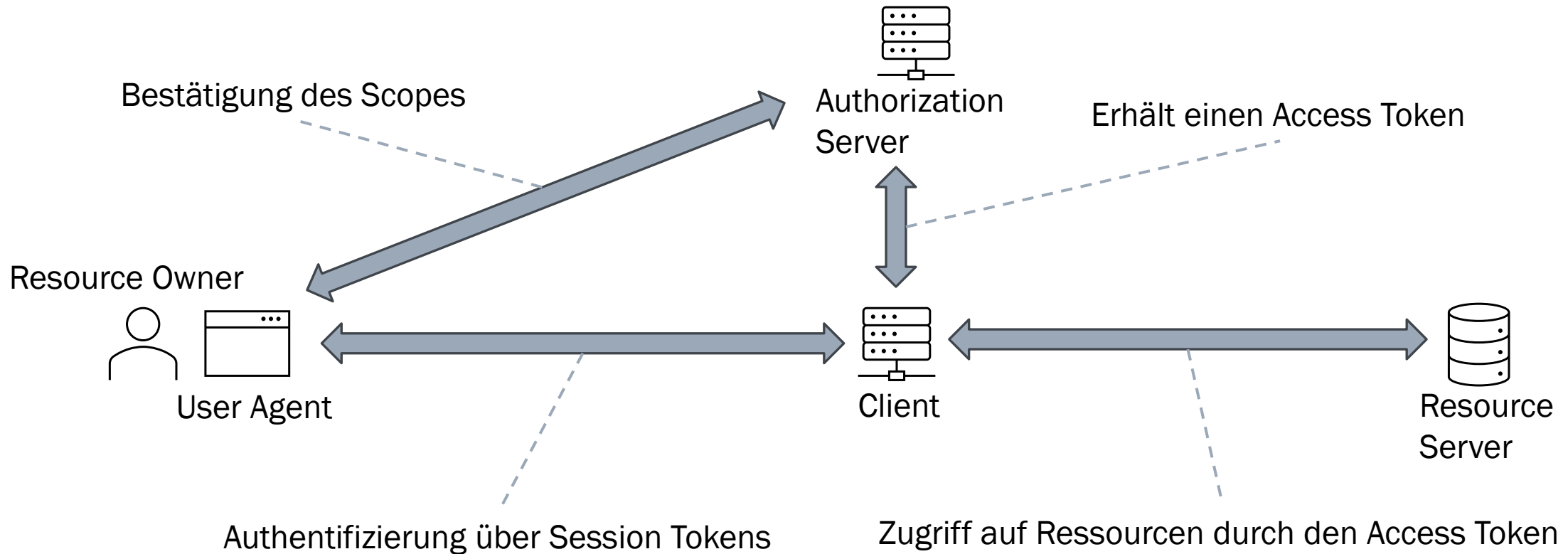
Authorization Server

- Server der Autorisierungs- und Authentifizierungsanfragen des Clients verarbeitet
- Wenn der Prozess erfolgreich ist, wird ein **Access Token** gewährt (**Authorization Grant**)
- Ein Access Token ist meistens auf einen bestimmten **Scope** ausgestellt. Der bestimmt, auf welche Ressourcen zugegriffen werden kann

Flow

- Autorisierungs- und Authentifizierungsprozesse werden Flows genannt

Aufbau



Tokens

Access Token

- Gewährt dem Besitzer Zugriff auf den Resource Server
- Sind mit einem Scope assoziiert
- Lebensdauer in Minuten / Stunden

Refresh Token

- Mit einem Refresh Token kann man Abgelaufene Access Tokens erneuern
- Benötigen spezielle Bestätigung des Nutzers
- Lebensdauer in Tagen / Wochen

Identity Token

- Ähnlich wie ein Access Token, dienen aber ausschließlich zur Identifikation und Authentifizierung von Nutzern
- Lebensdauer in Stunden

Session Token

- Nicht aus dem OAuth Protokoll, dienen zur Absicherung der Schnittstelle Browser - Client
- Lebensdauer in Stunden

Token Formate

Opaque (en: undurchsichtige) Tokens

- Im Token stehen keine Informationen
- Wird Üblicherweise von OAuth verwendet
- Um den Scope oder andere Informationen zu ermitteln, muss man den Userinfo- oder Introspection-Endpoint aufrufen
- Vorteil: Erhöhte Sicherheit durch undurchsichtigkeit
- Nachteil: erhöhter Aufwand zur Verifizierung von Tokens

JSON Web Tokens

- Benutzerinformationen und erfüllte Scopes sind im Token gespeichert
- Der Token wird mit dem Zertifikat des Authentication Servers signiert
- Vorteil: Der Resource Server kann den Token sofort verifizieren
- Nachteil: Wenn der Token über eine offene Leitung transportiert wird, kann man

Endpoints (HTTP-Schnittstellen)

Authorization Endpoint

- Die Autorisierungsanfrage landet hier
- Der Nutzer wird zur Anmeldung auf eine Seite des Authorization Servers umgeleitet, und dann zurück zur Anwendung. Dabei wird der Authorization Code Übergeben

Token Endpoint

- Man kann den Authorization Code gegen einen Access Token tauschen
- Man kann Refresh Tokens einlösen

Introspection Endpoint

- Man kann den Status eines Tokens einsehen
- Im Gegensatz zu Userinfo ist der Endpoint abgesichert (client_id und client_secret müssen mitgesendet werden)

Userinfo Endpoint

- Man kann die Benutzerinformationen zu einem Token einsehen
- Erweiterung durch OpenId Connect Standard

Revocation Endpoint

- Um bestehende Tokens aufzulösen, schickt man eine Anfrage an den Revokation Endpoint

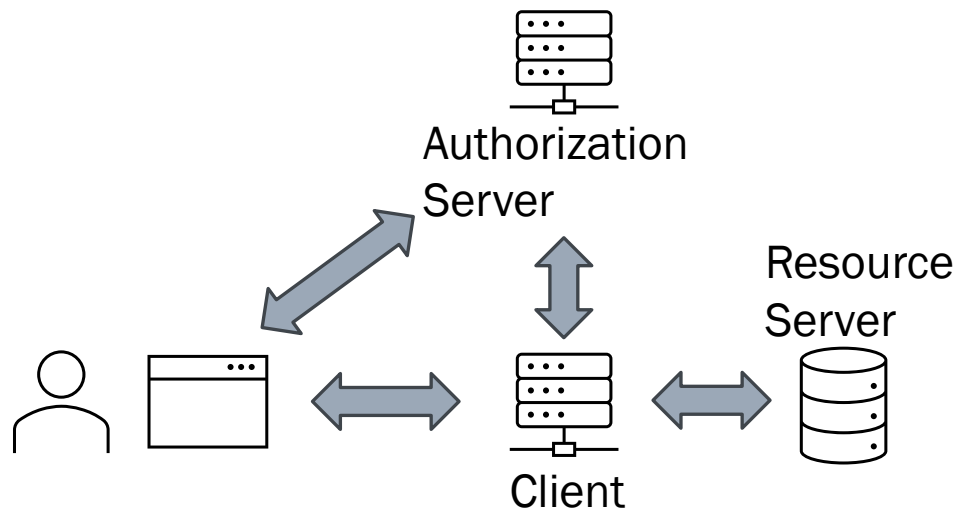
Redirection Endpoint

- Gehört dem Client
- Dient dazu, Rückmeldungen vom Authorization Server zu empfangen

Typen von Clients

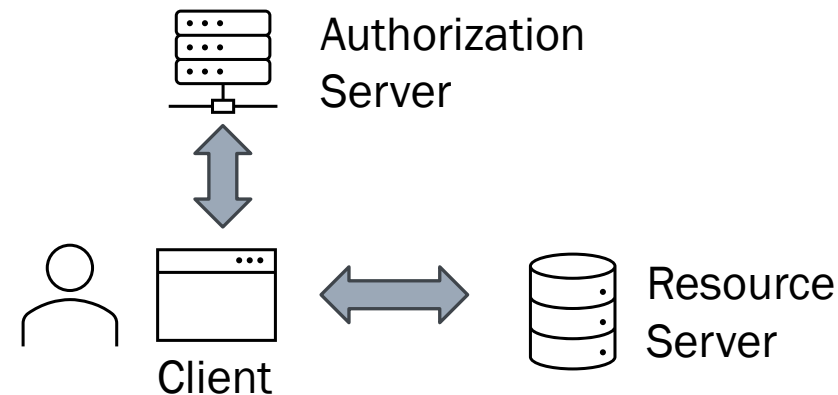
Confidential Client

- Client ist ein Web Server



Public Client

- Client ist eine Single Page Application (SPA), Mobile oder Desktop Anwendung



Wesentlicher Unterschied:

Der Vertrauliche Client ist in der Lage ein Client Geheimnis (sicher) zu speichern

Flows

Im Folgenden wird der Ablauf ausgewählter Flows detailliert ausgeführt. Vorher wird kurz der Zweck dieser Flows erklärt.

Als erstes wird der Implicit Flow vorgestellt, um die sichereren Varianten zu motivieren

Weitere Sicherheitsüberlegungen werden erst später behandelt

Implicit Flow

- Ziel: Erlangen von Access Token
- Für Public Clients gedacht
- Unsicher

Authorization Code Flow

- Verbesserte Version des Implicit Flow

Authorization Code Flow mit PKCE (Proof Key for Code Exchange)

- PKCE ist eine Erweiterung von OAuth2.0
- PKCE ist für Public Clients gedacht, bietet aber auch Sicherheitsvorteile für Confidential Clients

Token Revokation

- Ziel: Widerruf von Access und Refresh Tokens

Client Credentials Flow

- Ziel: Authentisierung von Services untereinander

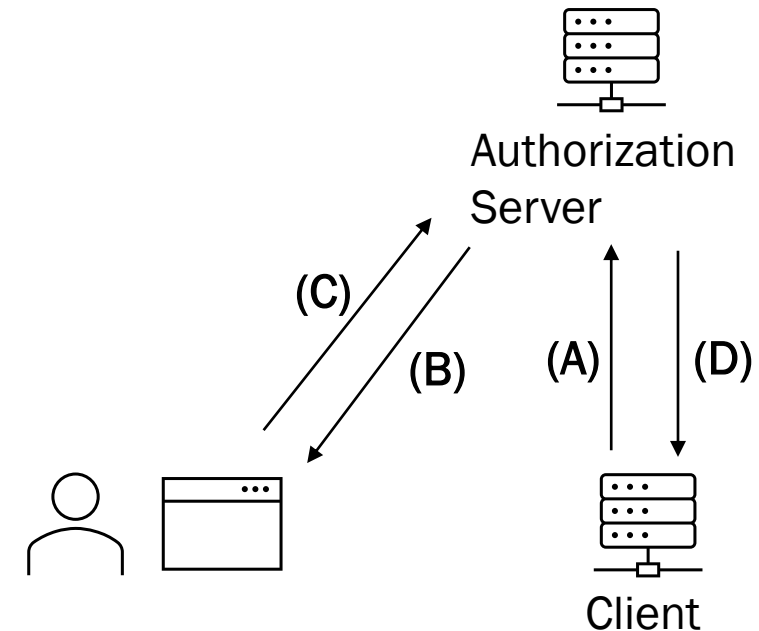
Implicit Flow

Der Client initialisiert den Implicit Flow, fragt den Authorization Endpoint an (A). Die Anfrage enthält schon die Redirect URI aus (D).

Der Authorization Server lässt die Anfrage vom Nutzer bestätigen. Das kann zum Beispiel per Redirect im Browser erfolgen (B).

Der Nutzer bestätigt die Authorisierung und stimmt zusätzlichen Scopes zu (C).

Der Authorization Server sendet den Access Token zurück an den Client (per Redirect URI vereinbart) (D).



Wesentliche Schwachstelle:

Token Leakage: Der Token wird per Redirect zurück an den Client gegeben, steht also zwischenzeitlich in der URL, und könnte so nach außen gelangen. (Der Client ist eine Website im Browser, und kann typischerweise keine Signale empfangen)

Authorization Code Flow

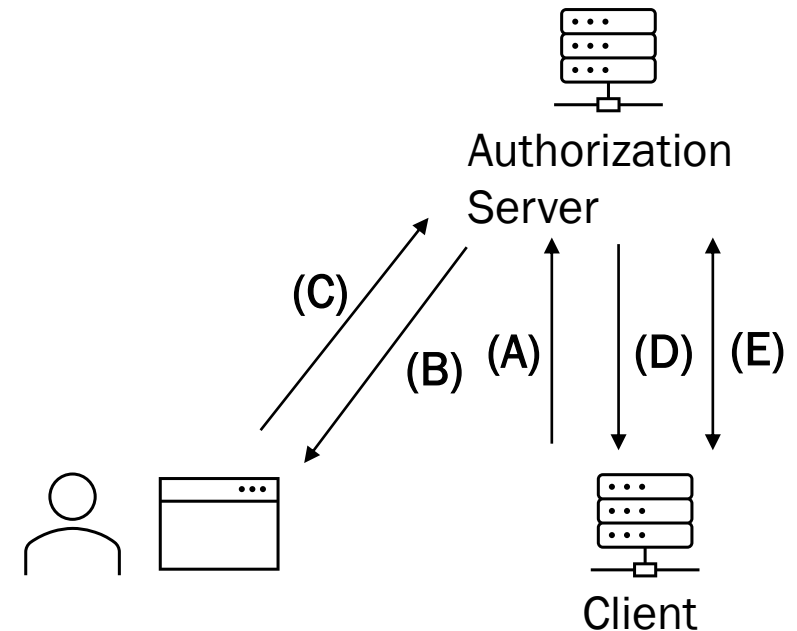
Der Client initialisiert den Flow, fragt den Authorization Endpoint an (A). Die Anfrage enthält schon die Redirect URI aus (D).

Der Authorization Server lässt die Anfrage vom Nutzer bestätigen. Das kann zum Beispiel per Redirect im Browser erfolgen (B).

Der Nutzer bestätigt die Authorisierung und stimmt zusätzlichen Scopes zu (C).

Der Authorization Server sendet einen sehr kurzlebigen Authorization Code zurück an den Client (D).

Der Client tauscht dann den Authorization Code eigenständig gegen den Access Token ein (E).



Sicherheitstechnische Überlegungen zum Authorization Code Flow

Der Access Token wird über eine Sichere Leitung übertragen (TLS)

- Und kann nicht mehr durchsickern

Eignet sich nicht für Public Clients

- Da der Client sich eigenständig an den Authorization Server zurückmeldet, muss er sich mit einem Client Secret authentifizieren

Eine mögliche Erweiterung des Flows sind „state“ Parameter

- Die Anfrage an den Authorization Server wird um einen zufälligen String (=„state“) erweitert. Der Server sendet denselben „state“ dann zurück. So kann der Client verschiedene Schritte verschiedener Authorisierungsprozesse einander zuordnen und verifizieren

Authorization Code Flow mit PKCE (Proof Key for Code Exchange)

Ursprünglich für Public Clients gedacht, um den Authorization Code Flow auch ohne Client Secret abzusichern. Wird aber auch für Confidential Clients empfohlen.

Der Client generiert einen (kryptographisch zuverlässig) zufälligen String (= Code-Verifier)



Aus dem Code-Verifier wird ein weiterer String durch eine Hashfunktion (sha256) generiert (= Code)



In der ersten Anfrage an den Authorization Server wird der Code als Parameter mitgesendet



In der zweiten Anfrage wird dann sowohl der Code Verifier als auch der Code mitgesendet

So kann der Authorization Server verschiedene Anfragen einander zuordnen. Unterschied „state“ Parameter: „state“ schützt den Client, PKCE schützt den Authorization Server

Revokation Flow

- Der Client kann die Autorisierung frühzeitig beenden, zum Beispiel wenn der Resource Owner sich ausloggt. Dafür muss der Client authentifiziert sein.
- Ausgestellte Tokens können aber nicht vom Resource Owner selbst widerrufen werden.
- Refresh Tokens können widerrufen werden
- Access Tokens können nur dann widerrufen werden, wenn sie Opaque sind

Client Credentials Flow

- Confidential Clients müssen sich mit anderen Servern austauschen
- Dabei client_id und client_secret zu senden ist eine unnötige Schwachstelle
- Der Client erfragt also einen Client Credentials Grant vom Authorization Server, und bekommt einen Access Token zurück
- Mit diesem Token können die Clients sich Identifizieren, ohne vorher Geheimnisse auszutauschen. Es reicht das beide Parteien dem Authorization Server vertrauen

Angriffsvektoren und Gegenmaßnahmen

Im Folgenden werden
ausgewählte Angriffe gegen
OAuth2.0 behandelt, und Ansätze
zur Abwehr vorgestellt



CSRF Attacken (Cross-Site Request Forgery)



Authorization Code Injection



Phishing



Replay Attacken

CSRF-Attacken im Allgemeinen

Definition (synopsys.com)

Cross-Site Request Forgery (CSRF) is an attack that forces authenticated users to submit a request to a Web application against which they are currently authenticated

Beispiel einer CSRF-Attacke

Der Benutzer wird über einen Sessiontoken zum Webserver authentifiziert, der als Cookie gespeichert ist

Der Angreifer bringt den Nutzer dazu, auf ein böses Formular zu klicken

Das Formular sendet eine böse Anfrage an den Webserver. Die wird akzeptiert, weil sie vom authentisierten Browser aus gesendet wird

Abwehr mit CSRF-Tokens

Der Webserver integriert CSRF-Tokens (Random Strings) in das Formular. (<input type=hidden >)

Beim Absenden werden die CSRF-Tokens zurück an den Webserver gesendet und können abgeglichen werden

Ein Fremdes Formular würde nicht über diese Tokens verfügen, und der Angriff würde erkannt

Token Injection mit CSRF (beim Authorization Code Flow)

- Hier wird die vertrauliche Beziehung User Agent (Browser) und dem Authorisierungsserver ausgenutzt

Der Angreifer bringt den Benutzer dazu, auf einen Bösartigen Link zum Redirect Endpoint des Ziel-Clients zu betätigen.

Dabei wird ein Authorization Code benutzt, den der Angreifer vorher selbst vom Authorization Server angefragt hat

Der Client speichert den Access Token ab

Der Useragent ist jetzt mit dem Session Token gegenüber dem Client authentisiert, aber gegenüber dem Resource Server als **jemand anderes eingeloggt**

Wenn jetzt geheime Daten eingegeben werden, kann der Angreifer Die mitlesen

Gegenmaßnahmen gegen CSRF / Token Injection

- Austausch und Vergleich vom „state“ Parameter

Der Angreifer bringt den Benutzer dazu, auf einen Bösartigen Link zum Redirect Endpoint des Ziel-Clients zu betätigen.

Dabei wird ein Authorization Code benutzt, den der Angreifer vorher selbst vom Authorization Server angefragt hat

Der Client merkt, dass die Rückmeldung keinen Gültigen „state“ Parameter enthält

- Erweiterte Form von Token Injection: Der Angreifer fängt einen Authorization Flow ab, und gelangt so an einen validen „state“ Parameter. Jetzt führt er selbst den Flow aus, und gibt dann den Authorization Code zurück an den Client. Um solche Angriffe zu verhindern, muss PKCE implementiert werden

OAuth Phishing

- Man initiiert einen Authorization Flow beim Benutzer, der einen bösartigen Client autorisiert (die aber wie ein bekannter Client aussieht)

Prävention

- Auf dem OAuth Server keine fremden Clients zulassen
- Markierung verdächtiger Clients beim Autorisierungs-Prompt

Replay Attacken

- Eine Replay Attacke besteht dagegen darin, die verschlüsselten Nachrichten mitzuschreiben und zu einem späteren Zeitpunkt wieder abzuspielen

TLS / SSL Verbindung

- Das TSL-Protokoll schützt gegen Replay Attacken durch zufällige Werte im Handshake

Nonce („number used once“)

- Ein Zufälliger Nonce-Wert wird mitgesendet, Der Empfänger merkt sich die Nonces
- Wenn ein Nonce wiedererkannt wird, ist die Anfrage ungültig.
- $\text{Dauer(Nonce)} > \text{Dauer(Access Token)}$

Implementation

Kurze Vorstellung von
Möglichkeiten, OAuth aufzusetzen

Öffentliche Anbieter (oft Soziale Netzwerke)



- Google
- Github

+ Die meisten Benutzer sind schon „Registriert“
– Nur Authentifizierung Möglich

Cloud Anbieter



- Okta IAM
- Microsoft Entra ID
- Duo Security

+ Jemand Anderes hat die Verantwortung

Self Hosted



- Identity Server
- Keycloak
- Authentik

+ Volle Kontrolle für Konfiguration und Erweiterung

Dokumentation

Das gesamte Protokoll ist vollständig und offen von der IETF (Internet Engineering Task Force) veröffentlicht

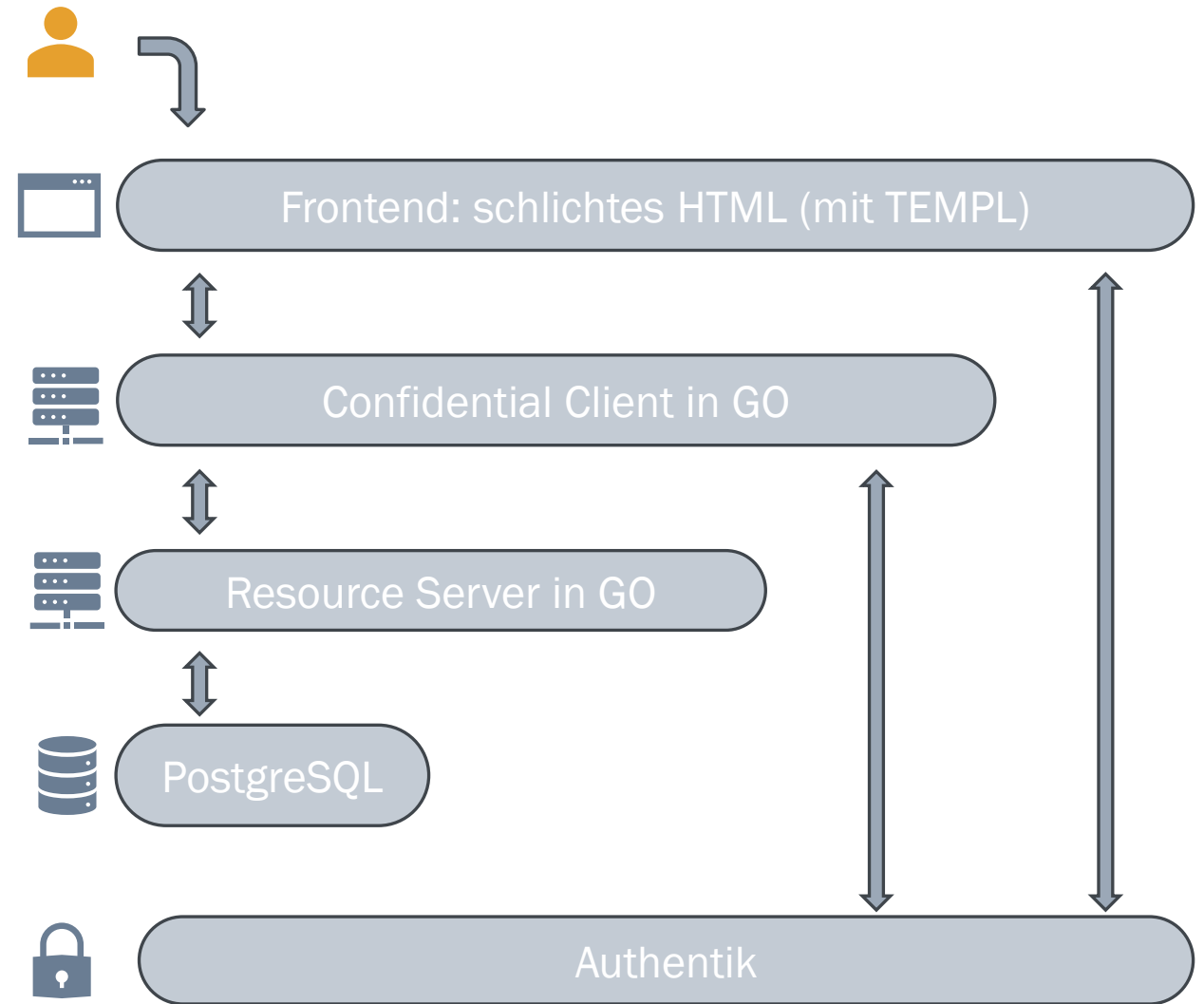
Wesentlich sind RFC6749 (OAuth 2.0) und RFC7636 (PKCE)

Anbieter haben aber oft noch Nebenbedingungen oder Erweiterungen, erwarten aber gleichzeitig Vorwissen über das Protokoll

Persönlich kann ich die Seite von Okta empfehlen, selbst wenn man einen anderen Provider benutzt

Eigene Implementation: Notiz Anwendung

- ✓ Notiz-API als Resource-Server
- ✓ Autorisierung mit PKCE und state
- ✓ CSRF-Tokens im Frontend
- ✓ TLS



Erweiterungen / Alternativen zu OAuth

OpenId
Connect

OAuth
2.1, PKCE

SAML
(Security Assertion Markup
Language)

Token
Binding

Vielen Dank für
Ihre
Aufmerksamkeit
